МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1

Выполнил:
Студент группы ИБМ3-34Б

Анфиногенова М. А.
Преподаватель:
Правидина А. Д.,

Бурмистрова М. В.

1) **Задание:**

Разработать программы для решения биквадратного уравнения.

1. Программы должна быть разработаны в виде консольного приложения на языке Python. Разработайте три программы на языке Python:
   i. с применением процедурной парадигмы (использование функций, оператор def)
   ii. с применением объектно-ориентированной парадигмы (использование классов)
   iii. с применением функциональной парадигмы (сопоставление с образцом)
2. Каждая программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

2.1) Линейная парадигма

```
import sys
import math


args = sys.argv[1:]

#  Ввод коэффициента A
a = None
if len(args) > 0:
    try:
        a = float(args[0])
    except ValueError:
        print(f"Значение '{args[0]}' из командной строки некорректно — "
              f"введите коэффициент с клавиатуры.")
while a is None:
    try:
        a = float(input("Введите A: "))
    except ValueError:
        print("Некорректный ввод. Ожидается действительное число. Повторите.")
```

```
#  Ввод коэффициента B
b = None
if len(args) > 1:
    try:
        b = float(args[1])
    except ValueError:
        print(f"Значение '{args[1]}' из командной строки некорректно — "
              f"введите коэффициент с клавиатуры.")
while b is None:
    try:
        b = float(input("Введите B: "))
    except ValueError:
        print("Некорректный ввод. Ожидается действительное число. Повторите.")

#  Ввод коэффициента C
c = None
if len(args) > 2:
    try:
        c = float(args[2])
    except ValueError:
        print(f"Значение '{args[2]}' из командной строки некорректно — "
              f"введите коэффициент с клавиатуры.")
while c is None:
    try:
        c = float(input("Введите C: "))
    except ValueError:
        print("Некорректный ввод. Ожидается действительное число. Повторите.")

#  Решение биквадратного уравнения
if a == 0:
    print("Коэффициент A не может быть равен 0 — уравнение не биквадратное.")
else:
    d = b * b - 4 * a * c
    roots = []

    if d < 0:
        roots = []
    elif d == 0:
        t = -b / (2 * a)
        if t < 0:
            roots = []
        elif t == 0:
            roots = [0.0]
        else:
            roots = [math.sqrt(t), -math.sqrt(t)]
    else:
        t1 = (-b + math.sqrt(d)) / (2 * a)
        t2 = (-b - math.sqrt(d)) / (2 * a)
```

```
        for t in (t1, t2):
            if t < 0:
                continue
            elif t == 0:
                roots.append(0.0)
            else:
                roots.append(math.sqrt(t))
                roots.append(-math.sqrt(t))

    #  Вывод результата
    if not roots:
        print("Действительных корней нет.")
    else:
        unique = sorted(set(roots))  # убираем дубли за счет множества и сортируем по возрастанию
        print("Действительные корни:")
        for r in unique:
            print(f"  x = {r:g}")
```



2.2) Процедурная парадигма

```
import sys
import math


def get_coefficient(prompt, cli_value=None):
    """Запрашивает коэффициент. Сначала проверяет значение из командной строки,
    при некорректности — переходит к вводу с клавиатуры."""
    if cli_value is not None:
        try:
            return float(cli_value)
        except ValueError:
            print(f"Значение '{cli_value}' из командной строки некорректно — "
                  f"введите коэффициент с клавиатуры.")

    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Некорректный ввод. Ожидается действительное число. Повторите.")


def compute_discriminant(a, b, c):
```

```
    """Дискриминант квадратного уравнения относительно t = x²."""
    return b * b - 4 * a * c


def roots_from_t(t):
    """Возвращает действительные x по найденному t = x²."""
    if t < 0:
        return []
    if t == 0:
        return [0.0]
    return [math.sqrt(t), -math.sqrt(t)]


def solve_biquadratic(a, b, c):
    """Возвращает список действительных корней x.
    Если a = 0, уравнение не биквадратное — возвращает None."""
    if a == 0:
        return None

    d = compute_discriminant(a, b, c)

    if d < 0:
        return []

    if d == 0:
        t = -b / (2 * a)
        return roots_from_t(t)

    t1 = (-b + math.sqrt(d)) / (2 * a)
    t2 = (-b - math.sqrt(d)) / (2 * a)
    return roots_from_t(t1) + roots_from_t(t2)


def print_result(roots):
    """Выводит результат решения на экран."""
    if roots is None:
        print("Коэффициент A не может быть равен 0 — уравнение не биквадратное.")
        return
    if not roots:
        print("Действительных корней нет.")
        return
    unique = sorted(set(roots))  # убираем дубли и сортируем по возрастанию
    print("Действительные корни:")
    for r in unique:
        print(f"  x = {r:g}")


def main():
    args = sys.argv[1:]
```

```
    a_cl = args[0] if len(args) > 0 else None
    b_cl = args[1] if len(args) > 1 else None
    c_cl = args[2] if len(args) > 2 else None

    a = get_coefficient("Введите A: ", a_cl)
    b = get_coefficient("Введите B: ", b_cl)
    c = get_coefficient("Введите C: ", c_cl)

    roots = solve_biquadratic(a, b, c)
    print_result(roots)


if __name__ == "__main__":
    main()
```

```
C:\Users\Маша\PycharmProjects\pythonProject\venv\Scripts\python.exe D:\Maria_Anf\Парадигма\Lab1\Lab1\lab1_proc.py
Введите A: 4
Введите B: 7
Введите C: 8
Действительных корней нет.

Process finished with exit code 0
```

2.3) ООП парадигма

```
import sys
import math


class CoefficientReader:
    """Читает коэффициенты: из командной строки или с клавиатуры."""

    def read(self, prompt, cli_value=None):
        if cli_value is not None:
            try:
                return float(cli_value)
            except ValueError:
                print(f"Значение '{cli_value}' из командной строки некорректно — "
                      f"введите коэффициент с клавиатуры.")
        while True:
            try:
                return float(input(prompt))
            except ValueError:
                print("Некорректный ввод. Ожидается действительное число. Повторите.")


class BiquadraticEquation:
    """Биквадратное уравнение ax^4 + bx^2 + c = 0."""

    def __init__(self, a, b, c):
        self.a = a
        self.b = b
```

```
        self.c = c

    def discriminant(self):
        """Дискриминант квадратного уравнения относительно x^2."""
        return self.b ** 2 - 4 * self.a * self.c

    def roots_from_t(self, t):
        """Корни x, соответствующие значению t = x^2."""
        if t < 0:
            return []
        if t == 0:
            return [0.0]
        return [math.sqrt(t), -math.sqrt(t)]

    def solve(self):
        """Возвращает список действительных корней x.
        Если a = 0, уравнение не биквадратное — возвращает None."""
        if self.a == 0:
            return None

        d = self.discriminant()

        if d < 0:
            return []

        if d == 0:
            t = -self.b / (2 * self.a)
            return self.roots_from_t(t)

        t1 = (-self.b + math.sqrt(d)) / (2 * self.a)
        t2 = (-self.b - math.sqrt(d)) / (2 * self.a)
        return self.roots_from_t(t1) + self.roots_from_t(t2)


class BiquadraticConsoleApp:
    """Консольное приложение: связывает ввод и решение."""

    def __init__(self):
        self.args = sys.argv[1:]

    def command_line_arg(self, index):
        """Возвращает аргумент командной строки по индексу или None."""
        if len(self.args) > index:
            return self.args[index]
        return None

    def run(self):
        reader = CoefficientReader()
```

```
        a = reader.read("Введите A: ", self.command_line_arg(0))
        b = reader.read("Введите B: ", self.command_line_arg(1))
        c = reader.read("Введите C: ", self.command_line_arg(2))

        equation = BiquadraticEquation(a, b, c)
        roots = equation.solve()

        if roots is None:
            print("Коэффициент A не может быть равен 0 — уравнение не
биквадратное.")
            return

        if not roots:
            print("Действительных корней нет.")
            return

        print("Действительные корни:")
        for r in sorted(set(roots)):
            print(f"  x = {r:g}")


if __name__ == "__main__":
    BiquadraticConsoleApp().run()
```

